

INFORME DE TALLER PRÁCTICO EXPERIMENTAL

DATOS DE LA ASIGNATURA	
Nombre del Estudiante: Michael Arteaga	Nivel: Segundo
Carrera: Desarrollo de Software	Docente: Msc Zapata Yanez Veronica Marcela
Asignatura: Aplicaciones Web I	Periodo académico: octubre 2025 – marzo 2026

1. TEMA DEL TALLER

Creación de una interfaz intuitiva y eficiente que facilite la búsqueda y selección de productos.

2. DESCRIPCIÓN DEL TALLER

Crear una interfaz que facilite la búsqueda, exploración y selección de productos por parte de los usuarios, basándose en la presentación y navegación en los productos para agregar y seleccionar los productos a un carrito de compras

3. RESULTADO DE APRENDIZAJE ATADO AL TALLER PRÁCTICO EXPERIMENTAL

El taller consiste en el diseño y desarrollo de un sitio web orientado a la venta de productos, por lo tanto en el desarrollo de la web encontraremos al momento:

1. Buscar productos mediante una barra de búsqueda.
2. explorar productos organizados por categoría.
3. visualizar información del producto.
4. seleccionar el producto y agregarlo a un carrito de compras.

La pagina web fue echa aplicando HTML, CSS, JAVASCRIPT y lo aprendido en clases con un diseño responsivo para que se adapte a todos los dispositivos.

4. FUNDAMENTACIÓN TEÓRICA DE LA PRÁCTICA

- Previamente se creo la pagina index implemente y cree una pagina de admin en la que existe un formulario para poder agregar nuevos productos mediante la implementación de javascript

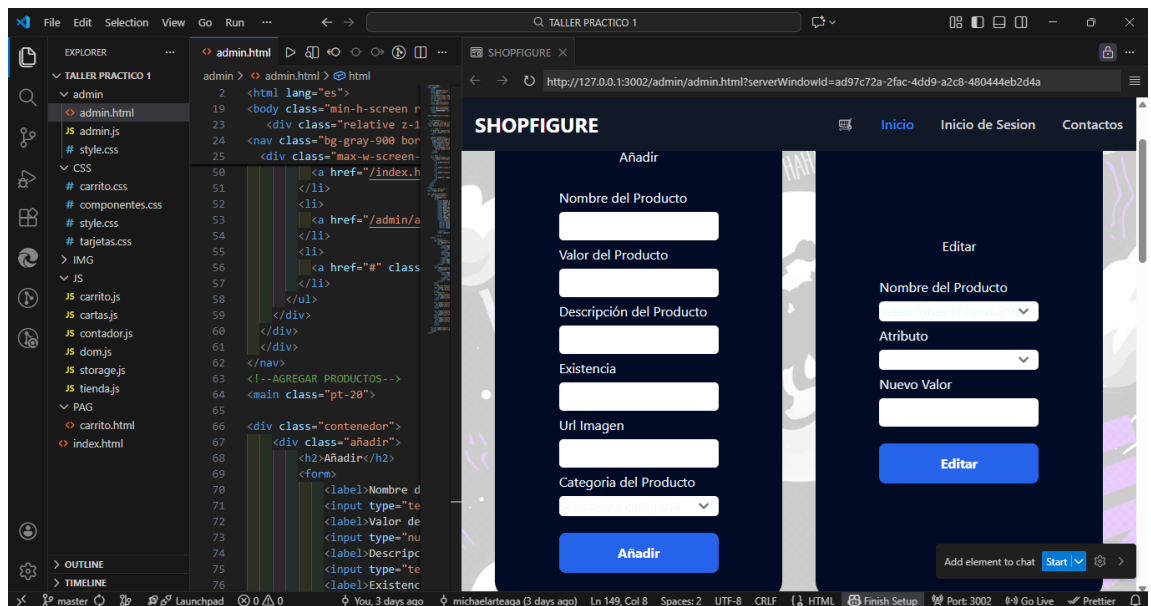
```
<!--AGREGAR PRODUCTOS-->
<main class="pt-20">
•
•
<div class="contenedor">
•   <div class="añadir">
•       <h2>Añadir</h2>
•       <form>
•           <label>Nombre del Producto</label>
•           <input type="text" id="productoAñadir"
name="nombredelProducto">
•           <label>Valor del Producto</label>
•           <input type="number" id="valorAñadir">
•           <label>Descripción del Producto</label>
```

```

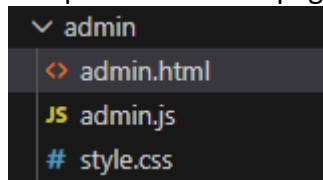
•      <input type="text" id="descripcionAñadir">
•      <label>Existencia</label>
•      <input type="number" id="existenciaAñadir">
•      <label >Url Imagen</label>
•      <input type="text" id="ImagenAñadir">
•      <label>Categoria del Producto</label>
•      <select id="categoriaAñadir">
•          <option>Selecciona categoría</option>
•          <option value="onepiece">One Piece</option>
•          <option value="dragonball">Dragon Ball</option>
•          <option value="bleach">Bleach</option>
•          <option value="marvel">Marvel</option>
•
•      </select>
•      <input class="button" type="button" id="botonAñadir"
value="Añadir">
•
•      </form>
•  </div>
•  <!--EDITAR-->
•  <div class="editar">
•      <h2 >Editar</h2>
•      <form>
•          <label>Nombre del Producto</label>
•          <select id="productoEditar">
•              <option value="">Selecciones el
Producto</option>
•              </select>
•          <label>Atributo</label>
•          <select id="atributoEditar">
•              <option value=""></option>
•          </select>
•          <label>Nuevo Valor</label>
•          <input type="text" id="nuevoAtributo">
•          <input class="button" type="button"
id="botonEditar" value="Editar">
•          </form>
•      </div>
•  <div class="eliminar">
•      <h2>Eliminar</h2>
•      <form >
•          <label>Nombre del Producto</label>
•          <select id="productoEliminar">
•              <option value="">Seleccione el Producto</option>
•          </select>
•          <input class="button" type="button"
id="botonEliminar" value="Eliminar">
•          </form>
•      </div>
•  </div>
•  <!--MOSTRAR MENSAJE-->

```

```
• <div class="contenedorMensaje">
•   <div id="mensaje"></div>
• </div>
• <!--PRODUCTOS-->
• <div class="contenedorProductos">
•   <h2 class="text-3xl font-bold text-center mb-8">Figuras
de One Piece</h2>
•   <div class="mostrarProductos" data-
categoria="onepiece"></div>
• </div>
•
•   <div class="contenedorProductos">
•   <h2 class="text-3xl font-bold text-center mb-8">Figuras
de Dragon Ball</h2>
•   <div class="mostrarProductos" data-
categoria="dragonball"></div>
• </div>
•
•   <div class="contenedorProductos">
•   <h2 class="text-3xl font-bold text-center mb-8">Figuras
de Bleach</h2>
•   <div class="mostrarProductos" data-
categoria="bleach"></div>
• </div>
•   <div class="contenedorProductos">
•   <h2 class="text-3xl font-bold text-center mb-8">Figuras
de Marvel</h2>
•   <div class="mostrarProductos" data-
categoria="marvel"></div>
• </div>
• </main>
•
• <script type="module" src="/admin/admin.js"></script>
• <script
src="https://cdnjs.cloudflare.com/ajax/libs/flowbite/2.2.1/flowbite
.min.js"></script>
• </body>
```



la misma es responsiva y contiene las categorías de agregar, editar y eliminar complementando la pagina con su propio css y la parte de javascript.



En el javascript tenemos las las clases y selectores que fuimos viendo en clases:

```
inicializarStorage();
```

```
let productos = obtenerProductos();
```

```
//CONSTANTES QUE SE UTILIZARAN EN LAS FUNCIONES QUE VAMOS A UTILIZAR  
EN ADMIN
```

```
//YA SEA PARA AÑADIR, EDITAR, ELIMINAR NUESTROS PRODUCTOS
```

```
/*AÑADIMOS UN MENSAJE PARA LOS CONDICIONALES */
```

```
const mensaje = document.getElementById("mensaje");
```

```
// AÑADIR
```

```
const añadirProducto = document.getElementById("productoAñadir");
```

```
const añadirValor = document.getElementById("valorAñadir");
```

```
const añadirExistencia = document.getElementById("existenciaAñadir");
```

```
const añadirDescripcion = document.getElementById("descripcionAñadir");
```

```
const añadirImagen = document.getElementById("imagenAñadir");
```

```
const añadirCategoria = document.getElementById("categoriaAñadir");
```

```
// EDITAR
```

```
const productoEd = document.getElementById("productoEditar");
```

```
const atributoEd = document.getElementById("atributoEditar");
```

```
const nuevoAtributoEd = document.getElementById("nuevoAtributo");
```

```
// ELIMINAR
```

```
const productoE = document.getElementById("productoEliminar");
```

//FUNCIONES

```
function mostrarMensaje(clase) {  
  mensaje.className = "";  
  mensaje.classList.add(clase);  
  setTimeout(() => mensaje.className = "", 2500);  
}
```

```
function recargar() {  
  guardarProductos(productos);  
  setTimeout(() => location.reload(), 1200);  
}
```

// AÑADIR PRODUCTO

```
document.getElementById("botonAñadir")?.addEventListener("click", (e) => {  
  e.preventDefault();
```

```
// CREAMOS LA CONTANTE DE NUEVO PRODUCTO DENTRO DEL MISMO ABRA LOS  
  CAMPOS ID , NOMBRE PRECIO, STOCK, DESCRIPCION, IMAGEN, CATEGORIA
```

```
// LA CATEGORIA DE MOMENTO SE DIVIDE EN TRES
```

```
const nuevoProducto = {  
  id: Date.now(),  
  nombre: añadirProducto.value.trim(),  
  precio: Number(añadirValor.value),  
  stock: Number(añadirExistencia.value),  
  descripcion: añadirDescripcion.value.trim(),  
  imagen: añadirImagen.value.trim(),  
  categoria: añadirCategoria.value  
};
```

```
// CONDICIONALES PARA NO LOS CAMPOS
```

```
// SI ESTAN VACIOS ALGUNOS CAMPOS
```

```
if (Object.values(nuevoProducto).some(v => !v)) {  
  mostrarMensaje("llenarCampos");  
  return;  
}
```

```
// SI YA TENEMOS EL PRODUCTO
```

```
if (productos.some(p => p.nombre === nuevoProducto.nombre)) {  
  mostrarMensaje("repetidoError");  
  return;  
}
```

```
//UNA VES LLENADOS LOS CAMPOS SE REALIZA NOS DARA UN MENSAJE DE  
  REALIZADO
```

```
productos.push(nuevoProducto);  
mostrarMensaje("realizado");  
recargar();
```

```
});
```

//EDITAR PRODUCTO

```
document.getElementById("botonEditar")?.addEventListener("click", (e) => {
    e.preventDefault();

    const nombre = productoEd.value;
    const atributo = atributoEd.value;
    let nuevoValor = nuevoAtributoEd.value;

    const producto = productos.find(p => p.nombre === nombre);

    if (!producto || !atributo || !nuevoValor) {
        mostrarMensaje("llenarCampos");
        return;
    }

    if (atributo === "precio" || atributo === "stock") {
        nuevoValor = Number(nuevoValor);
    }

    producto[atributo] = nuevoValor;
    mostrarMensaje("realizado");
    recargar();
});
```

// ELIMINAR PRODUCTO

```
document.getElementById("botonEliminar")?.addEventListener("click", (e) => {
    e.preventDefault();

    const nombre = productoE.value;
    const index = productos.findIndex(p => p.nombre === nombre);

    if (index === -1) {
        mostrarMensaje("noExisteError");
        return;
    }

    productos.splice(index, 1);
    mostrarMensaje("realizado");
    recargar();
});

/* =====
DOMContentLoaded es un evento del navegador que se dispara cuando Todo el
HTML ya fue cargado y convertido en DOM
el Dom es la representacion del html en carpetas
===== */

window.addEventListener("DOMContentLoaded", () => {
```

```
productoEd.innerHTML = "";
productoE.innerHTML = "";
atributoEd.innerHTML = "";

productos.forEach(p => {
  productoEd.innerHTML += `<option
value="${p.nombre}">${p.nombre}</option>`;
  productoE.innerHTML += `<option
value="${p.nombre}">${p.nombre}</option>`;
});

if (productos.length > 0) {
  Object.keys(productos[0]).forEach(key => {
    if (key !== "id") {
      atributoEd.innerHTML += `<option value="${key}">${key}</option>`;
    }
  });
}

renderProductos(productos, { mostrarBotones: false });
});
```

- Implementar en el head del proyecto en la pagina index los archivos css y js de las páginas que utilizaremos para guiarnos y que sean de ayuda para nuestros proyectos de página. En nuestro caso usaremos FLOWBITE Y GITHUB para tener nuestro proyecto en la nube de github.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <!--FLOWBITE-->
  <link
href="https://cdn.jsdelivr.net/npm/flowbite@4.0.1/dist/flowbite.min.css"
rel="stylesheet" />
  <script
src="https://cdn.jsdelivr.net/npm/flowbite@4.0.1/dist/flowbite.min.js"></
script>
  <script src="https://cdn.tailwindcss.com"></script>
  <!--GITHUB-->
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link rel="stylesheet" href="CSS/style.css">
  <title>SHOPFIGURE</title>
</head>
```

- Desarrollamos nuestra Navbar mezclando elemento y buscando una buena

utilidad de la misma al ser una pagina web de compras se implemento una barra de búsqueda

```
<body class="relative min-h-screen">

  <!-- FONDO CON IMAGEN -->
  <div
    class="absolute inset-0 bg-cover bg-center"
    style="background-image:
url('https://4kwallpapers.com/images/walls/thumbs_3t/16712.jpg');">
  </div>

  <!-- CAPA DE DESVANECIDO -->
  <div class="absolute inset-0 bg-white/80"></div>
  <!-- usa bg-black/60 si lo quieres oscuro -->

  <!-- CONTENIDO -->
  <div class="relative z-10">
    <nav class="bg-gray-900 border-gray-700 fixed top-0 left-0 w-full z-50 ">
      <div class="max-w-screen-xl flex flex-wrap items-center justify-between
mx-auto p-4">

        <!-- LOGO -->
        <a href="#" class="flex items-center space-x-3">
          <span class="self-center text-2xl font-bold text-
white">SHOPFIGURE</span>
        </a>

        <!--BOTON DE BUSQUEDA-->
        <form class="mt-4 md:mt-0 md:ml-6" role="search">
          <div class="relative">
            <input type="search" placeholder="Buscar figuras..."
            class="block w-full md:w-64 p-2 pl-10 text-sm text-white border border-
gray-600 rounded-lg bg-gray-800 focus:ring-blue-500 focus:border-blue-
500"
            />
            <svg class="absolute left-3 top-2.5 w-4 h-4 text-gray-400"
            xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24"
            stroke="currentColor">
              <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
              d="M21 21l-4.35-4.35M17 11a6 6 0 11-12 0 6 6 0 0112 0z" />
            </svg>
          </div>
        </form>

        <!-- BOTON RESPONSIVO-->
        <button data-collapse-toggle="navbar-default" type="button"
          class="inline-flex items-center p-2 w-10 h-10 justify-center text-
sm text-gray-400 rounded-lg md:hidden hover:bg-gray-700 focus:outline-
none focus:ring-2 focus:ring-gray-600"
          aria-controls="navbar-default" aria-expanded="false">
          <span class="sr-only">Abrir menú</span>
          <svg class="w-5 h-5" aria-hidden="true"
            xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 17 14">
```



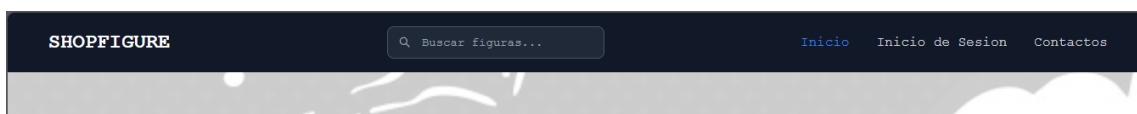
```

    <path stroke="currentColor" stroke-linecap="round" stroke-
linejoin="round" stroke-width="2"
      d="M1 1h15M1 7h15M1 13h15" />
  </svg>
</button>

<!-- MENU -->
<div class="hidden w-full md:block md:w-auto" id="navbar-default">
  <ul class="font-medium flex flex-col p-4 md:p-0 mt-4 border border-
gray-700 rounded-lg bg-gray-800 md:flex-row md:space-x-8 md:mt-0
md:border-0 md:bg-gray-900">
    <li>
      <a href="#" class="block py-2 px-3 text-white bg-blue-600
rounded md:bg-transparent md:text-blue-500 md:p-0">Inicio</a>
    </li>
    <li>
      <a href="#" class="block py-2 px-3 text-gray-300 rounded
hover:bg-gray-700 md:hover:bg-transparent md:hover:text-blue-500 md:p-
0">Inicio de Sesion</a>
    </li>
    <li>
      <a href="#" class="block py-2 px-3 text-gray-300 rounded
hover:bg-gray-700 md:hover:bg-transparent md:hover:text-blue-500 md:p-
0">Contactos</a>
    </li>
  </ul>
</div>

</div>
</nav>
<main class="pt-20">
<div>

```



A la barra de búsqueda del index se le dio funcionalidad con javascript

```

// =====
// BUSCADOR DE PRODUCTOS
// =====
const buscador = document.getElementById("buscador");

if (buscador) {
  buscador.addEventListener("input", () => {
    const texto = buscador.value.toLowerCase();
    const tarjetas = document.querySelectorAll(".product-card");

    tarjetas.forEach(card => {
      const nombre = card
        .querySelector("h3")

```

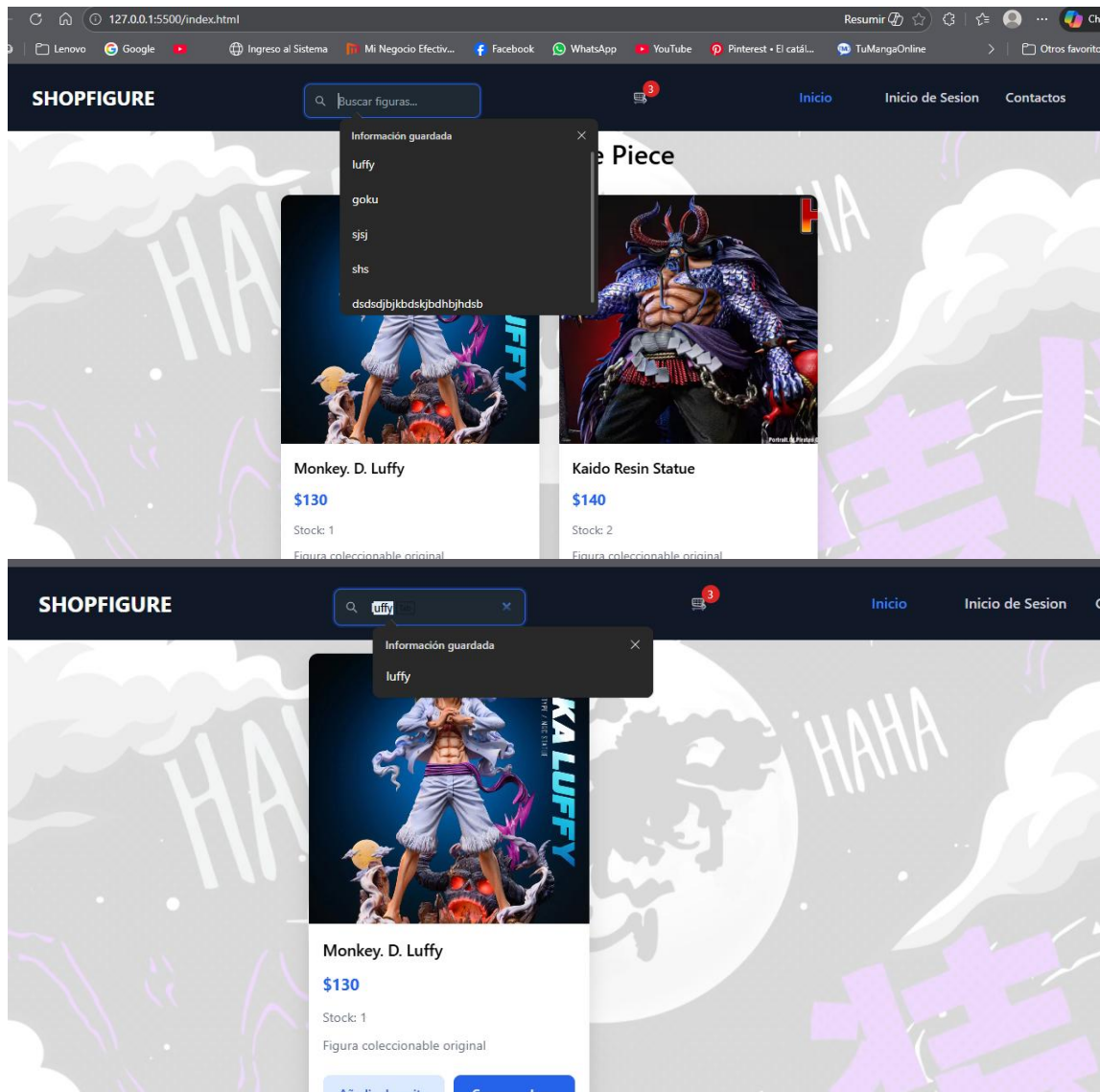
```

    .textContent
    .toLowerCase();

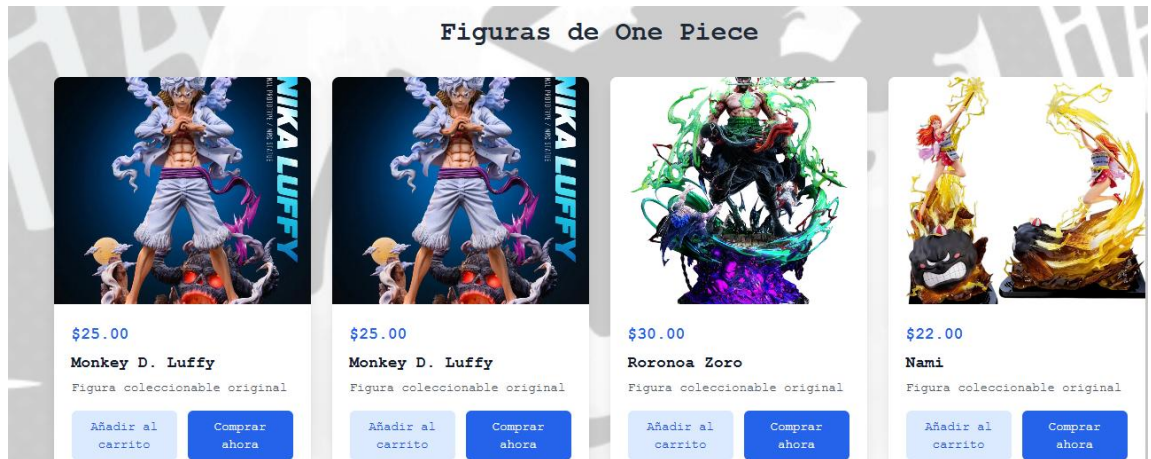
    if (nombre.includes(texto)) {
      card.style.display = "block";
    } else {
      card.style.display = "none";
    }
  });
});
}

```

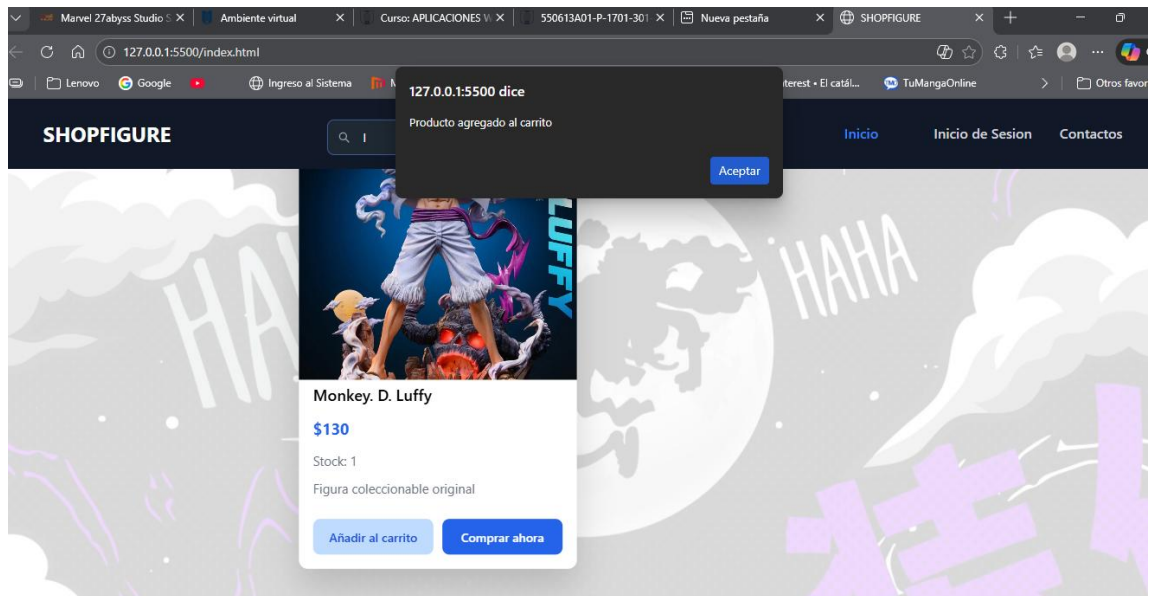
Haciendo la barra funcional.



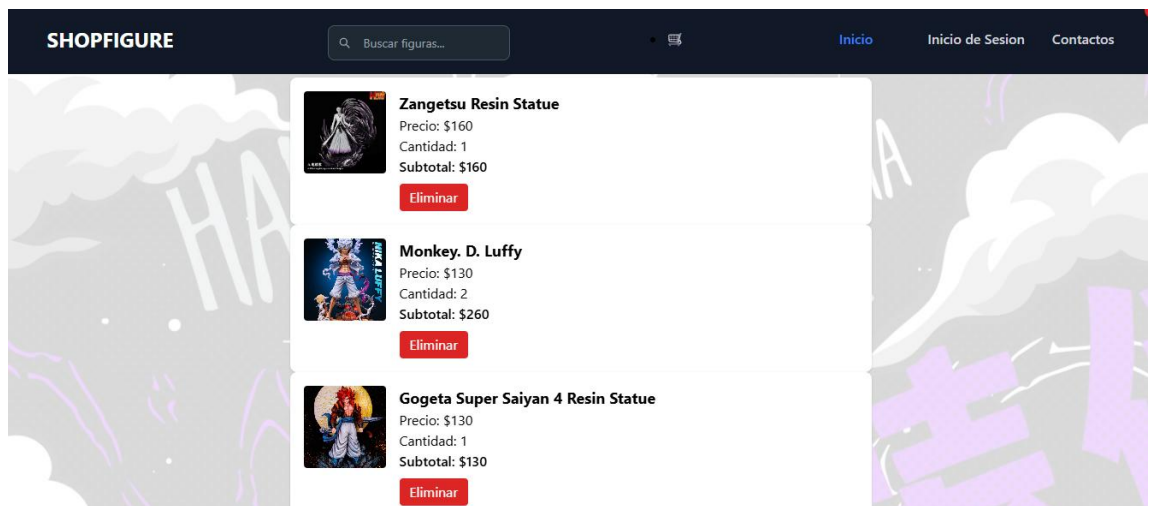
- Desarrollamos las tarjetas de los productos y vamos puliendo detalles con flowbite y css, y desarrollando javascript para que las tarjetas se creen una vez de añadan en la pagina de admin

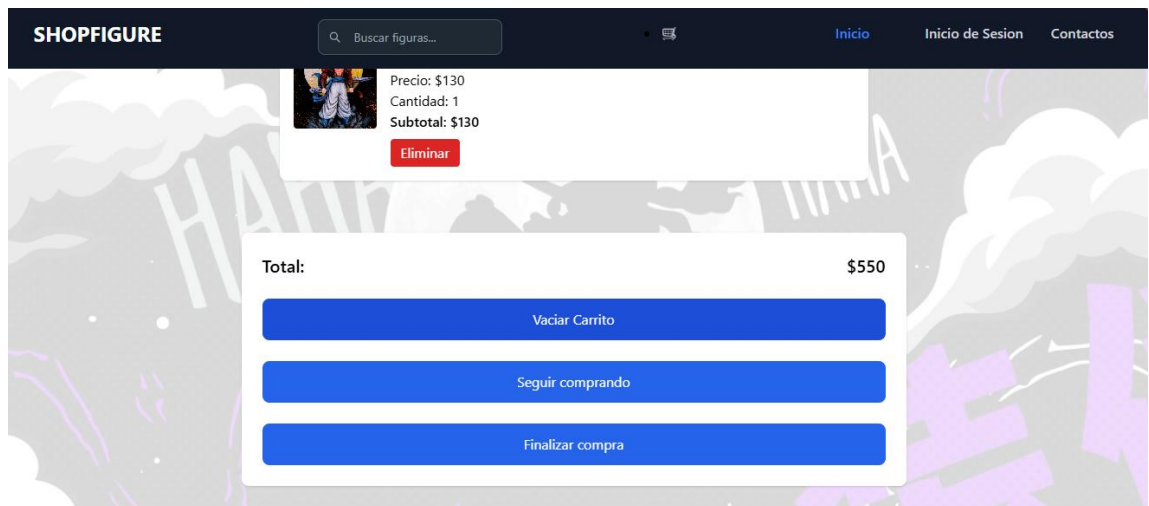


Asimismo, se utilizó **JavaScript** para gestionar la lógica de la aplicación, como la visualización dinámica de productos y la funcionalidad del carrito de compras. Para simular una base de datos, se empleó **LocalStorage**, permitiendo almacenar temporalmente los productos y el carrito sin necesidad de una base de datos real.



Se desarrollo la pagina del carrito y a su vez también se implemento javascript.





5. DESCRIPCIÓN DE LAS ACTIVIDADES DESARROLLADAS DURANTE LA PRÁCTICA.

TECNOLOGÍAS UTILIZADAS	
	• HTML5: Estructura del sitio web. • CSS / Tailwind CSS: Estilos y diseño responsivo. • Flowbite: Componentes visuales predefinidos. • JavaScript: Lógica de interacción, carrito de compras y almacenamiento. • GitHub: Control de versiones y respaldo del proyecto. • Visual Studio Code: Editor de código.

ACTIVIDAD	DESCRIPCIÓN
Instalación del entorno	Instalación de Visual Studio Code y extensiones necesarias
Creación de estructura	Organización de carpetas y archivos del proyecto
Desarrollo del HTML	Creación de páginas principales y estructura base
Implementación de CSS	Aplicación de estilos y diseño responsivo
Uso de Flowbite	Incorporación de componentes visuales
Desarrollo del carrito	Implementación del carrito usando JavaScript y LocalStorage
Pruebas	Verificación del funcionamiento en distintos navegadores

6. MATERIALES Y EQUIPOS

Corresponde a los materiales, equipos, insumos, maquinaria, instrumental, etc. disponibles en los laboratorios institucionales o escenarios de aprendizaje asignados.

MATERIAL / EQUIPO/ INSUMO	UTILIDAD
Laptop	Dispositivo usado para realizar todo el proceso de configuración del hardware a través del software PUTTY y los propios ajustes del sistema.
Visual Studio Code	Programa usado para escribir y ejecuta el texto de HTML que da forma a la web.

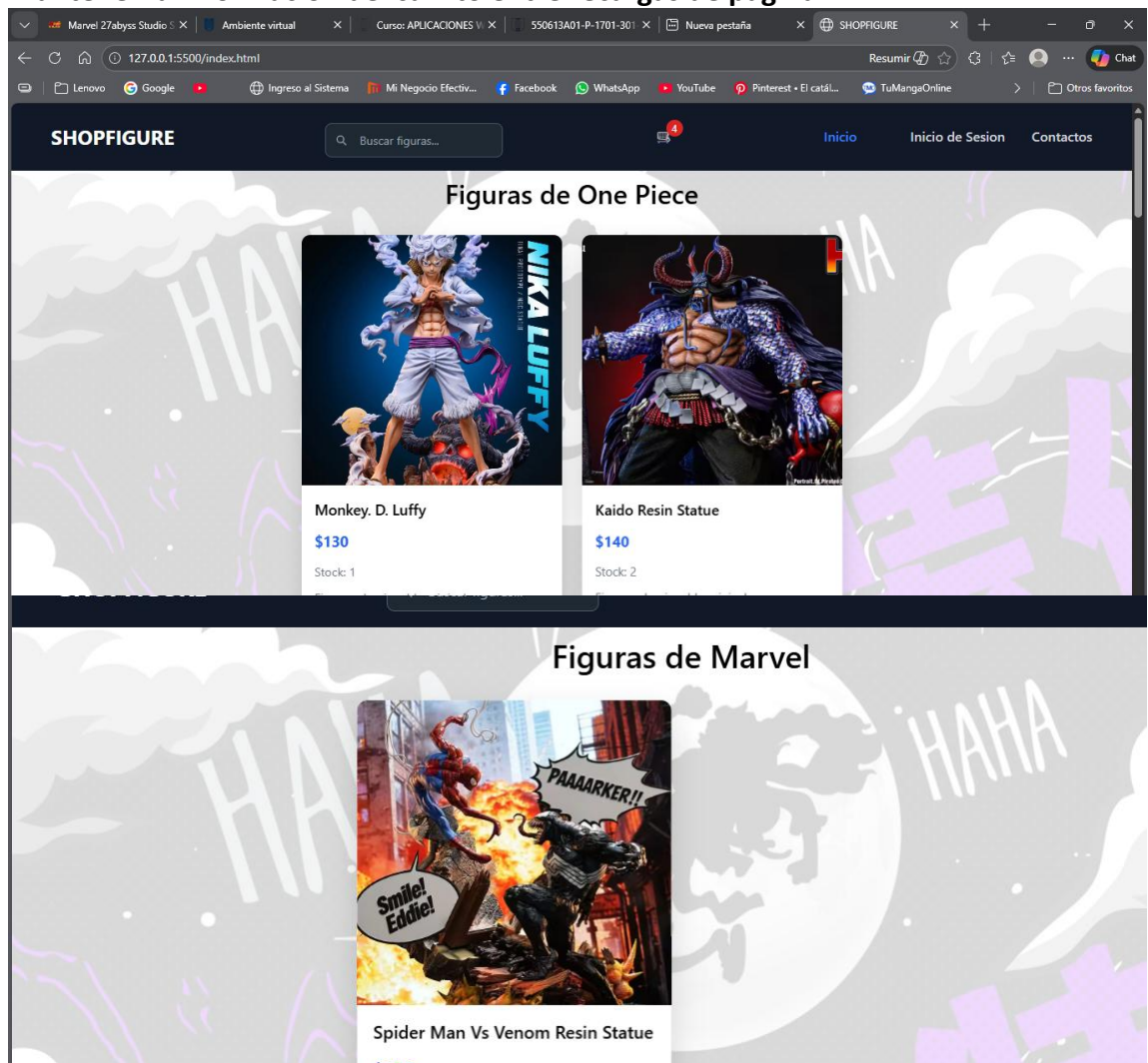
	HTML5	Lenguaje de Marcado de Hipertexto, usado para describir las características de la web y darle forma.
	Brave Browser	Navegador elegido para probar el funcionamiento de la web durante el desarrollo.
	PostImages	Web para el almacenamiento de imágenes que podemos incluir en nuestra página a través de los enlaces que nos da.
	Tailwind	Framework para agilizar y facilitar el diseño de nuestra web.
	JavaScript	Lógica de interacción y funcionalidad de la pagina

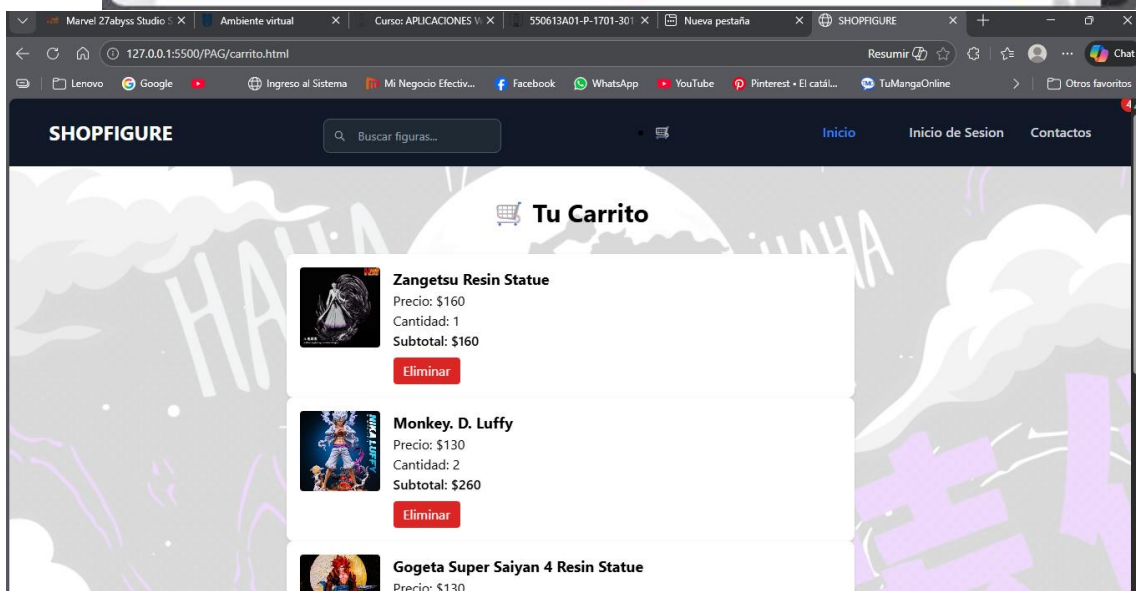
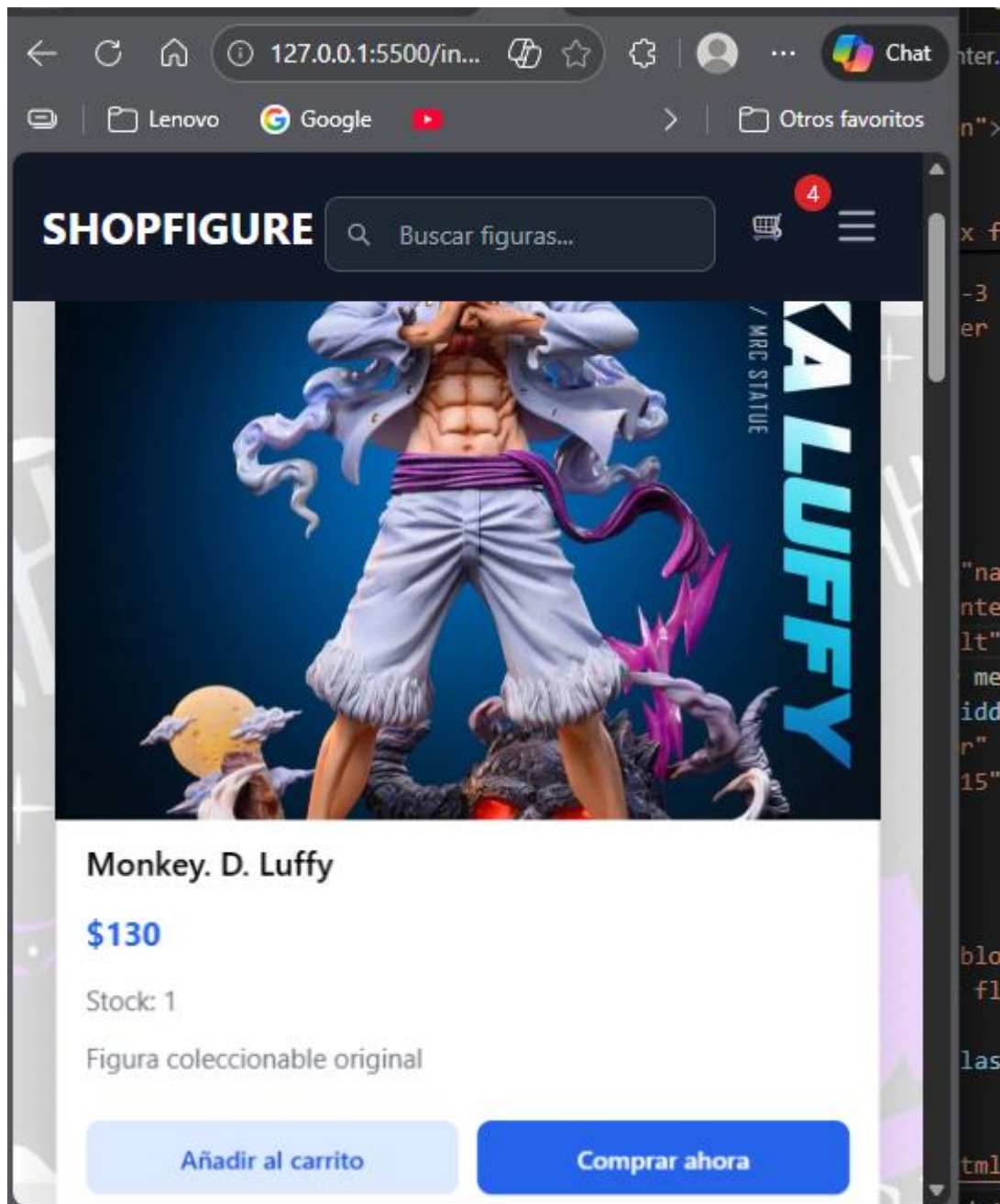
7. IDENTIFICACIÓN DE RIESGOS Y EQUIPOS DE PROTECCIÓN PERSONAL

ANÁLISIS DE RESULTADOS

El sistema permite:

- Visualizar tarjetas de productos dinámicamente.
- Agregar productos al carrito de compras.
- Almacenar los productos seleccionados de forma temporal.
- Mantener la información del carrito entre recargas de página.





La implementación del framework Flowbite junto con Tailwind CSS permitió desarrollar una interfaz moderna, clara y adaptable a distintos dispositivos.

El uso de JavaScript y LocalStorage facilitó la simulación de una base de datos, permitiendo una experiencia funcional similar a una tienda real sin requerir un backend.

8. CONCLUSIONES

- El uso de frameworks CSS modernos agiliza significativamente el desarrollo web.
- JavaScript permite implementar funcionalidades dinámicas como el carrito de compras.
- El uso de LocalStorage es una solución efectiva cuando no se dispone aún de una base de datos.
- El control de versiones con GitHub facilita la organización y recuperación del proyecto.

9. RECOMENDACIONES

- Investigar más sobre frameworks frontend modernos.
- Implementar una base de datos real en futuras versiones del proyecto.
- Continuar utilizando control de versiones durante el desarrollo.

10. Bibliografía

- Ariganello, E. (2020). *Redes Cisco: Guía de estudio para la certificación CCNA 200-301*. Madrid: RA-MA Editorial. Obtenido de https://elibro.net/es/ereader/itsqmet/222695?fs_q=ccna&prev=fs
- CISCO Networking Academy. (2025). *NETACAD*. Obtenido de <https://www.netacad.com/es/launch?id=ba7d6178-ad9f-4074-83a9-2abacbb9840d&tab=curriculum&view=8f9a5f26-8dd9-590b-9f94-3fc61f7a13d8>
- Cloudflare. (2025). *CLOUDFLARE*. Obtenido de <https://www.cloudflare.com/es-es/learning/ddos/glossary/open-systems-interconnection-model-osi/>
- Cloudflare. (2025). *CLOUDFLARE*. Obtenido de <https://www.cloudflare.com/es-es/learning/ddos/glossary/tcp-ip/>

Enlace GitHub:

<https://github.com/MichaelArteaga-urey/Tienda.git>